

Техническое задание на практику по модулю:

ПМ2. Прикладное программирование.

Дисциплина: Основы разработки на Windows Forms (C#)

1. Цель проекта

Разработать полноценное настольное приложение на языке C# (технология Windows Forms), демонстрирующее навыки проектирования интерфейсов, работы с базами данных, сетевыми протоколами и асинхронным программированием.

2. Общие требования

- **Стек технологий:** C#, .NET Framework 4.8+ или .NET 6/8.
- **Тип приложения:** WinForms (Desktop).
- **Стабильность:** Корректная обработка исключений (try-catch), отсутствие критических сбоев при неверном вводе данных.
- **Сборка:** Наличие исполняемого файла (.exe) или инсталлятора (.msi).
- **Документация:** Описание структуры базы данных в виде ER-диаграммы.

3. Технические требования (обязательные модули)

3.1. Работа с данными

- **База данных:** Использование MySQL, SQLite или другой СУБД. Структура должна соответствовать 3НФ (третьей нормальной форме).
- **CRUD:** Реализация полного цикла операций (создание, чтение, обновление, удаление записей).
- **Файловая система:** Реализация экспорта или импорта данных в форматы JSON, XML или CSV (на выбор).

3.2. Интерфейс (UI/UX)

- **Интуитивность и дружелюбность:** Интерфейс должен быть понятным без инструкции. Расположение элементов должно соответствовать логике действий пользователя. Понятная навигация, наличие главного меню (MenuStrip) и контекстных меню.
- **Цветовая гамма:** использовать не более 3–4 основных цветов. Текст должен быть легко читаемым (высокий контраст между шрифтом и фоном).

- **Элементы управления:** Использование стандартных (DataGridView, TreeView, TabControl, ComboBox) и продвинутых компонентов.
- **Динамика:** Визуальное отображение процессов (использование ProgressBar, Label или NotifyIcon).
- **Адаптивность:** Корректное расположение элементов при изменении размеров окна (использование Anchor, Dock, TableLayoutPanel).

3.3. Логика и внешние интеграции

- **Сетевое взаимодействие:** Использование HttpClient для работы с внешними API (погода, курсы валют, поиск фильмов/книг и др.).
- **Асинхронность:** Выполнение тяжелых операций в фоне (async/await) во избежание блокировки интерфейса.
- **Таймеры:** Применение класса Timer для обновления данных в реальном времени, проверки дедлайнов или управления игровой логикой.

4. Рекомендуемые направления проектов

Студент может выбрать тему из списка ниже или предложить свою:

- **Информационные системы:** Мониторинг финансов/криптовалют, фитнес-клуб, планировщик задач (Канбан), учет сотрудников, библиотека.
- **Сетевые сервисы:** Новостной агрегатор, погодный клиент, поисковик рецептов/фильмов через API, P2P-чат.
- **Игровые проекты:** Морской бой (сетевой), Тетрис, Сапер, Змейка, Пятнашки, Арканойд.
- **Утилиты:** Конвертер файлов, приложение для шифрования (AES), менеджер заметок, калькулятор калорий.

5. Регламент и этика

- **Командная работа:** Допускается работа в группах по 2-3 человека (при условии разделения зон ответственности в коде).
- **Использование ИИ:** Допускается как вспомогательный инструмент. **Важно:** на защите студент должен уметь объяснить работу любого участка кода. Бездумное копирование сгенерированного кода без понимания принципов его работы ведет к снижению оценки.
- **Публикация:** Приветствуется загрузка проекта на GitHub.